RECUR

The Stripe for Solana

Solana Colosseum Frontier Hackathon
April 6 – May 11, 2026

> *Recur uses SPL Token delegation and automated Keepers to enable true "set-it-and-forget-it" recurring billing. Users sign once and keep their funds liquid. Merchants get automated USDC revenue, reliable webhooks, and a 3-line React SDK. We bring seamless Web2 SaaS billing to Web3 — non-custodial, borderless, and open.*

Table of Contents

1. Executive Summary	3
2. The Problem We Solve	3
3. Technical Architecture	4
4. Engineering Roadmap	5
5. Competitive Landscape	7
6. Our Structural Edges	9
7. Business Model & Revenue Validation	10
8. Verdict & Funding Narrative	12

1. Executive Summary

Recur is the first Solana-native, open-source, developer-first subscription billing engine. It brings Stripe-like recurring payment infrastructure natively to Solana — without centralised intermediaries, geographic restrictions, or capital lockups.

<$0.001	400ms	1 Tx	195+
per transaction	finality	user signs once, ever	countries, permissionless

2. The Problem We Solve

Web3 subscriptions are fundamentally broken in three ways:

- **Every 30 days users must manually approve a new transaction. Most don't. This creates catastrophic monthly churn that no SaaS or game studio can build a business on.** Manual re-signing:
- **Escrow-based solutions exist but force users to pre-lock large amounts of funds — destroying the utility of holding liquid USDC.** Capital lockup:
- **There is no open-source, embeddable React component a Solana game developer can drop in to monetise their players with recurring billing. Nobody has built this.** No developer primitive:

Stripe proved the market is real in October 2025 — they launched USDC subscription billing for recurring payments. But their version is US-only, EVM-only (Base and Polygon), requires fiat settlement into a bank account, and doesn't touch Solana at all. That gap is what Recur fills.

3. Technical Architecture

How it works — end to end

Recur's architecture has four layers that work together seamlessly:

Layer	What it does	Stack
Smart Contract	On-chain PDA stores subscriber + merchant + plan + timestamps. process_payment enforces time-lock and executes CPI transfer. cancel_subscription closes PDA and refunds SOL rent.	Anchor / Rust
Keeper Network	Off-chain cron runs every 60s. Pre-flight checks balance + delegation. Constructs and signs process_payment transaction with priority fee. Exponential backoff on failure.	Node.js / TypeScript

Merchant Dashboard	Wallet-login dashboard for plan creation, SDK snippet generation, webhook URL configuration, and live MRR analytics. Data from Supabase — zero RPC rate limit exposure.	Next.js + Supabase
Developer SDK	Single npm-installable <RecurButton /> component. Atomically signs SPL Token Approve + initialize_subscription in one transaction. Handles loading states and error toasts automatically.	React Native / npm

Key security properties

- **The Keeper can only call process_payment. It cannot move funds arbitrarily — it can only execute what the user previously delegated via SPL Token Approve.** Non-custodial:
- **User USDC remains in their wallet until billing date. No lockup, no escrow, no upfront capital commitment.** Capital-efficient:
- **Any developer globally can deploy plans. No KYC, no company registration, no Stripe account required.** Permissionless:
- **The Anchor program is public and auditable. No closed proprietary billing logic.** Open-source:

4. Engineering Roadmap

Module 1 — Smart Contracts (Anchor / Rust)

Goal: Build the secure, time-locked on-chain state and execution logic. All other modules depend on this completing first.

Task 1.1 — Define the PDA Structure

Create the Subscription struct with these fields:

- subscriber (Pubkey) — The wallet paying.
- merchant (Pubkey) — The recipient of funds.
- mint (Pubkey) — The SPL Token mint (e.g. USDC).
- amount (u64) — Amount to pull per interval.
- interval (i64) — Billing period in seconds.
- last_pull_timestamp (i64) — Clock time of last successful payment.
- bump (u8) — PDA derivation bump.

Task 1.2 — initialize_subscription Instruction

Creates the Subscription PDA. Validates constraints (amount > 0, minimum 1 USDC enforced). Initialises last_pull_timestamp to current on-chain clock. Called by the Frontend SDK immediately after the user signs the SPL Token Approve transaction. Gas Tank: merchant's Gas Tank wallet pays the ~0.002 SOL rent.

Task 1.3 — process_payment Instruction (Core Engine)

Called exclusively by the Keeper. Reads the Subscription PDA. Asserts current_time >= last_pull_timestamp + interval. Executes token::transfer_checked CPI using the delegated authority from the user's ATA to the merchant's ATA. Updates last_pull_timestamp to current_time. Includes a 60-second acceptable buffer window for clock drift.

Task 1.4 — cancel_subscription Instruction

Closes the PDA and refunds the ~0.002 SOL rent to the caller. Requires either subscriber or merchant signature. Triggers the Keeper to stop scheduling this subscription.

Module 2 — Backend & Keeper Network (Node.js / TypeScript)

Goal: Automate payments and sync on-chain data to off-chain infrastructure. Requires compiled Anchor IDL from Module 1.

Task 2.1 — Database Setup (Supabase / PostgreSQL)

Create tables for: Merchants, Plans, Subscriptions (synced with PDA state), and Webhook_Logs. All Keeper and Dashboard queries hit the database — zero direct RPC calls for read operations, eliminating rate limit exposure.

Task 2.2 — On-Chain Indexer

WebSocket listener on connection.onProgramAccountChange. Watches the Anchor program for new Subscription PDA creations. Inserts into Supabase immediately, so the Merchant Dashboard updates in real time.

Task 2.3 — Pre-Flight Cron Job

Runs every 60 seconds. Queries DB for subscriptions where next_billing_date <= NOW(). For each due subscription: calls getTokenAccountBalance to verify USDC balance and checks delegation status. If balance insufficient: skips transaction, logs failure, fires payment.failed webhook. If delegation revoked: calls force_cancel and fires subscription.canceled webhook.

Task 2.4 — On-Chain Executor

Constructs the process_payment transaction for all subscriptions that passed pre-flight. Signs with the server Keeper Keypair. Sends to the Anchor program with a dynamic priority fee. On failure: exponential backoff (10s wait, recalculate median priority fee, retry up to 3x).

Task 2.5 — Webhook Dispatcher

Listens for successful transaction signatures from the Executor. Sends HTTP POST to the merchant's registered endpoint with the full event payload.

Webhook payload example: { "event": "payment.success", "user": "7xKX...", "plan": "premium_pass", "amount": "5.00", "tx": "3xB..." }

Module 3 — Merchant Dashboard (Next.js)

Goal: Build the merchant-facing dashboard and the demo sandbox application. Requires Supabase DB URL and compiled contract IDL.

Task 3.1 — Dashboard Core

Wallet-connection login (Phantom / Backpack via @solana/wallet-adapter-react). Analytics page: MRR chart, active subscribers count, recent transaction history table. Stripe-quality UI with dark/light mode.

Task 3.2 — Plan & Webhook Management

Create Plan form: plan name, price, billing interval. SDK Generator: on save, displays a code block showing the merchant their ready-to-copy <RecurButton /> integration snippet. Webhook Settings: input field to register and test the merchant's webhook URL.

Task 3.3 — Demo Sandbox (Hackathon Critical)

A complete fake SaaS pricing page (e.g. "ItsU Premium Pass"). Beautiful pricing tiers with a working Subscribe button. The React SDK powers the subscription — the Demo shows the full end-to-end flow: merchant creates plan → user subscribes → Keeper pulls funds automatically → Dashboard MRR updates live. Billing interval set to 60 seconds for demo purposes.

Module 4 — Developer SDK (React / npm)

Goal: Package all complex on-chain logic into a single, embeddable React component that any developer can integrate in 3 lines of code.

Task 4.1 — The <RecurButton /> Component

An npm-installable React component that handles the entire subscription flow:

- Triggers Wallet Adapter signTransaction.
- Atomically packages two instructions into one transaction: TokenProgram.approve (grants the PDA the delegation allowance) and Program.initialize_subscription (creates the on-chain receipt PDA).
- Handles loading states, success toasts, and error handling — the merchant's frontend developer writes zero blockchain code.

Usage: <RecurButton planId="premium_pass" merchantWallet="7xKX..." /> — that is the complete integration.

5. Competitive Landscape

The market for recurring crypto payments is real, validated, and growing — but the Solana-native segment is almost entirely unserved. Here is every meaningful player in the space.

Competitor overview matrix

Product	Solana Native	Open Program	Game/dApp SDK	Borderless	Capital Free	Threat Level
Recur ✦	YES	YES	YES	YES	YES	US
Stripe Stablecoins	NO	NO	NO	NO	YES	HIGH
Loop Crypto	PARTIAL	NO	NO	YES	YES	HIGH
BoomFi	NO	NO	NO	YES	YES	MED
Streamflow	YES	YES	NO	YES	PARTIAL	MED
Zebec Network	ORIGIN	NO	NO	YES	PARTIAL	LOW
Clockwork (dead)	YES	YES	NO	YES	YES	GONE

Why every competitor fails the Solana developer

- **US businesses only. EVM only. Fiat settlement mandatory. No dApp/game integration. Requires a Stripe account and approval.** Stripe Stablecoins:
- **Closed managed service — they own the billing relationship. Solana added as an afterthought. No open Anchor program. No developer SDK.** Loop Crypto:
- **EVM-first. UAE regulatory overhead. Closed smart contract. Targets e-commerce, not protocol developers. No Solana program.** BoomFi:
- **Wrong primitive. Continuous token streaming locks capital. No merchant dashboard, no plan management, no webhooks. Built for payroll/vesting.** Streamflow:
- **Pivoted to enterprise ACH payroll. Building their own L3 chain (Nautilus). No subscription billing product. Not developer-accessible.** Zebec Network:
- **Defunct since Oct 2023. Left the gap Recur fills. Proved demand existed but failed to monetise it.** Clockwork:

6. Our Structural Edges

Edge 1 — Network: Solana vs EVM

Stripe runs on Base and Polygon. Loop's core is EVM. BoomFi is EVM-first. The economic difference is material at scale:

Metric	Polygon/Base (Stripe)	Solana (Recur)
Gas per Keeper tx	$0.05 – $2.00 (congestion)	< $0.001
Finality	~2 seconds (optimistic)	~400ms
Throughput	~2,000 TPS	65,000 TPS
Native dApp ecosystem	Fragmented across 300+ EVM chains	Single cohesive ecosystem

Edge 2 — Borderless vs US-only

Stripe's stablecoin subscriptions are available only to US-registered businesses, with mandatory fiat settlement into a traditional US bank account. This restriction is structural — tied to banking licenses and FinCEN compliance — and will not meaningfully change in the next 2–3 years.

Recur is permissionless. A developer in Nigeria, India, Argentina, or Vietnam deploys the Anchor program, registers a plan, and receives USDC directly to their Solana wallet — no bank, no KYC, no geo-block. Superteam, Recur's natural early community, operates in exactly these markets.

Edge 3 — Permissionless vs Closed API

Stripe, Loop, and BoomFi are all managed services: you apply, get approved, agree to terms, and they can suspend or terminate your merchant account at any time. If a country becomes sanctioned, your subscriptions stop.

Recur's Anchor program is a public on-chain primitive. The merchant owns their Subscription PDAs directly — Recur cannot de-platform a subscription, cannot freeze an account, cannot block a payment. The only authority over a subscription is the user who signed the SPL Token Approve.

Edge 4 — Game and dApp native

Every competitor is designed for Web2-style SaaS businesses with email notifications, KYC, and a bank account. Nobody has built a Phantom-native <RecurButton /> for Solana games.

Solana has hundreds of active games, DeFi protocols, and NFT platforms. Every single one of them has users they would charge recurring fees if a native billing primitive existed. Recur is that primitive.

7. Business Model & Revenue Validation

Recur has a dual-layer revenue model: a flat transaction fee that covers infrastructure economics, combined with a B2B SaaS tier that covers fixed costs and makes the protocol profitable from a small number of paying merchants.

Revenue Stream 1 — $0.05 Flat Fee per Transaction

Every time the Keeper fires a successful process_payment instruction, Recur charges the merchant $0.05 USD (deducted from the pulled amount or invoiced separately at scale).

> *Why flat, not percentage only: A 0.25% fee on a $1/month subscription is $0.0025. If Solana is slightly congested and the Keeper pays $0.01 in priority gas, the protocol would lose money on every execution. The $0.05 floor creates an unbreakable economic guarantee. Solana gas rarely exceeds a fraction of a cent — even during heavy congestion. Net profit per Keeper execution: approximately $0.04.*

Competitive validation of the flat fee:

- **charges 1% + $0.20 minimum per transaction for ACH direct debit. At $0.20 minimum, Recur at $0.05 is 4x cheaper on the fixed component alone.** GoCardless
- **charges $0.80 flat per ACH debit. Recur is 16x cheaper per execution.** Stripe ACH
- **does not publish flat fees — their pricing page refers only to percentage-based fees. Their EVM transaction costs (gas) are 50–2,000x higher than Solana, meaning their effective cost floor is far higher than ours.** BoomFi
- **requires a sales call for enterprise pricing, implying high minimums. Their infrastructure cost on EVM chains mirrors the same gas problem.** Loop Crypto

Verdict: The $0.05 flat fee is competitively aggressive and economically sound. It undercuts every traditional competitor by a factor of 4–16x while guaranteeing ~$0.04 net profit per execution regardless of network conditions.

Revenue Stream 2 — 0.25% Protocol Fee on Volume

In addition to the flat fee, Recur takes 0.25% of each payment processed.

Plan price	Stripe takes	Recur takes
$5 / month	$0.445 (2.9% + $0.30)	$0.0625 ($0.05 + 0.25%)
$10 / month	$0.59 (2.9% + $0.30)	$0.075 ($0.05 + 0.25%)
$50 / month	$1.75 (2.9% + $0.30)	$0.175 ($0.05 + 0.25%)

Recur is approximately 8–10x cheaper than Stripe at every price point. This is the pitch to every SaaS founder and game studio integrating billing: plug in Recur and immediately improve your unit economics.

Revenue Stream 3 — $49/month Pro Merchant Tier (The Linchpin)

The "valley of death" in transaction-fee protocols is fixed infrastructure cost. Processing $100,000 in monthly recurring volume at 0.25% generates $250 in protocol fees — which may not cover Helius RPC nodes, Supabase databases, and Vercel hosting.

> *The $49/month Pro Tier solves this directly. 15 to 20 Pro merchants cover the entire monthly infrastructure burn rate. Once servers are paid for, every cent of transaction volume becomes pure protocol profit.*

Pro Tier features (gated):

- CSV accounting exports for bookkeeping and tax.
- Any-to-Any token swap on subscription payment (pay in SOL, merchant receives USDC).
- Multi-team login and role management.
- Priority Keeper queue — Pro subscriptions are processed first in congestion.
- Advanced analytics: churn rate tracking, LTV per subscriber, cohort analysis.

Competitive validation of the SaaS tier:

- **(the subscription management platform Stripe integrates with) starts at $249/month. Our $49 Pro tier is 5x cheaper even before the transaction fee advantage.** Chargebee
- **(a Stripe competitor for SaaS) charges 5% + $0.50 per transaction with no flat-fee option. Their model requires massive volume to break even on infrastructure.** Paddle
- **requires enterprise sales calls — their effective minimum engagement is far higher than $49/month. We undercut them on both price and accessibility.** Loop Crypto

Verdict: The $49/month tier is not a stretch. It targets the exact demographic — SaaS founders and game studios already paying $249/month for Chargebee or $300+/month for Paddle — who would immediately switch to a $49/month crypto-native alternative that costs 10x less per transaction.

Economics at different volume milestones

Monthly Volume	0.25% Revenue	15 Pro Merchants	Total Monthly
$50,000	$125	$735	**$860**
$250,000	$625	$735	**$1,360**
$1,000,000	$2,500	$735	**$3,235**
$5,000,000	$12,500	$735	**$13,235**

The SaaS tier carries the protocol through the early low-volume phase. The transaction fee scales with ecosystem growth. The flat fee guarantees execution profitability. The three revenue streams are complementary and non-overlapping.

8. Verdict & Funding Narrative

The market gap is real, validated, and unoccupied on Solana

Stripe's October 2025 launch of stablecoin subscriptions on Base and Polygon is the single strongest external validation of this thesis. A $95 billion company just built this exact product — proving the demand is undeniable. They chose EVM chains. Solana is wide open.

Clockwork's 2023 shutdown (the only Solana-native automation protocol) proved that general-purpose automation without a monetisation model fails. Recur is not Clockwork — it is a specific billing product with three distinct revenue streams, a clear path to infrastructure self-sufficiency at 15–20 Pro merchants, and a price point that immediately undercuts every traditional competitor.

Why Colosseum Frontier should fund this

- **A working recurring billing protocol on Solana is usable by every game studio, DeFi protocol, and SaaS company in the ecosystem the day it ships. This is infrastructure, not speculation.** Real-world utility:

- **Four-module architecture with on-chain smart contracts, off-chain Keeper network, merchant dashboard, and open developer SDK. Each module has explicit dependencies and a working demo path.** Technical credibility:
- **Any judge — technical or non-technical — immediately understands the value proposition. "Stripe for Solana subscriptions" is a three-word pitch that needs no explanation.** Stripe comparison lands:
- **The $0.05 flat fee, 0.25% volume fee, and $49/month Pro tier are benchmarked against Stripe, Chargebee, Paddle, GoCardless, and BoomFi. All comparisons favour Recur by a factor of 4–16x.** Commercial model validated:

Why Superteam should be the distribution partner

Superteam's chapters are in India, Nigeria, Vietnam, Brazil, Turkey, and 10+ other markets where Stripe's US-only restriction is a permanent blocker. Every Superteam developer building a paid product has no native Solana subscription billing option. Recur is that option — permissionless, borderless, and deployable by any developer in any of those markets in under 10 minutes.

> *The opportunity: Stripe proved the market. Clockwork proved Solana needs this. Every competitor is either US-only, EVM-only, or built for the wrong use case. Recur is the Stripe for Solana — and it does not yet exist.*

Risk table

Risk	What it is	Our mitigation
Keeper centralisation	Single Keeper wallet is a centralised execution point that could go offline.	Keeper can only execute delegated payments — it cannot steal funds. V2 roadmap: permissionless Keeper network (same model as Chainlink).
Smart contract audit	Unaudited Anchor program for hackathon submission.	Devnet-only for hackathon. Mainnet launch gated on independent Ottersec or Neodyme audit.
Commercial viability	Clockwork failed to monetise Solana automation despite $4M raised.	Clockwork had no product revenue — only token. Recur has three distinct revenue streams, all modelled and benchmarked against live competitors.
Loop on Solana	Loop Crypto has partial SPL token support and could expand.	Loop is a closed managed service — they own the billing relationship. Recur's merchant owns their PDA. We also have an open game/dApp SDK that Loop has never built.